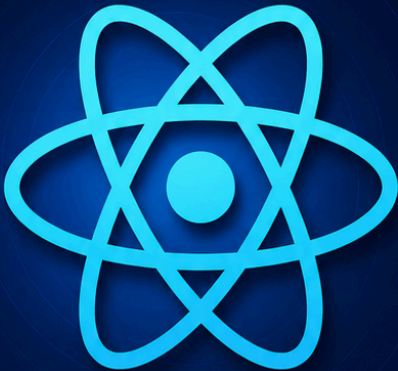


HOOKS Y CICLO DE VIDA EN REACT



Aprende a crear aplicaciones modernas y eficientes con los Hooks de **React** y comprende su ciclo de vida.

```
function App() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Contador: ${count}`;
  }, [count]);

  return <h1>Contador: {count}</h1>;
}
```






useState
Maneja el estado local.



useEffect
Efectos secundarios y ciclo de vida.



useContext
Comparte estado globalmente.



useRef
Referencias y valores persistentes.



useMemo
Optimiza cálculos costosos.



useCallback
Memoriza funciones y previene re-renders.



useReducer
Maneja estados complejos.

Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 3	Tema: Hooks y Ciclo de Vida en React
	Semana 11	Tutor: Ing. Victoria Castro

Introducción a React:

React es una biblioteca de JavaScript para construir interfaces de usuario basadas en **componentes**. Su filosofía principal se basa en la composición y el flujo de datos unidireccional.

- **Enfoque Declarativo:** Tú describes cómo quieres que se vea la interfaz según el estado, y React se encarga de actualizar el DOM de forma eficiente.
- **Virtual DOM:** React mantiene una representación en memoria de la UI. Cuando el estado cambia, compara el Virtual DOM con el real (proceso llamado *Reconciliation*) y solo actualiza lo estrictamente necesario.
- **Componentes Funcionales:** Actualmente son el estándar. Son funciones de JavaScript que devuelven JSX y utilizan Hooks para manejar la lógica.

Sección 1: Ciclo de Vida en React

El "Ciclo de Vida" se refiere a la serie de fases por las que pasa un componente desde que nace hasta que desaparece de la pantalla.

Las 3 Fases Principales:

1. **Montaje (Mounting):** El componente se crea y se inserta en el DOM.
2. **Actualización (Updating):** El componente se vuelve a renderizar debido a cambios en sus props o su state.
3. **Desmontaje (Unmounting):** El componente se elimina del DOM.

Equivalencias: De Clases a Hooks

Fase / Método de Clase	Hook Equivalente
componentDidMount	useEffect(() => { ... }, [])

Fase / Método de Clase	Hook Equivalente
componentDidUpdate	useEffect(() => { ... }, [dependencia])
componentWillUnmount	La función de retorno (<i>cleanup</i>) dentro de useEffect

Sección 2: Introducción a los Hooks

Los Hooks son funciones que te permiten "engancharte" a las características de React (estado, ciclo de vida) desde componentes funcionales. Se introdujeron en la versión 16.8 para evitar la complejidad de las clases (this, bind, etc.).

Las Reglas de Oro:

- **Solo en el nivel superior:** No llames Hooks dentro de bucles, condiciones o funciones anidadas.
- **Solo en funciones de React:** Úsalos solo en componentes funcionales o en tus propios *Custom Hooks*.

Sección 3: Manejo de Estado: useState

Es el Hook fundamental. Permite que tus componentes "recuerden" información.

```
const [estado, setEstado] = useState(valorInicial);
```

Concepto Clave: Cada vez que llamas a setEstado, React agenda un nuevo renderizado del componente. Si el nuevo valor es idéntico al anterior, React evitará el re-renderizado por optimización.

Sección 4: Efectos Secundarios: `useEffect`

Permite ejecutar código para interactuar con el "mundo exterior" (peticiones API, suscripciones, manipulación manual del DOM).

```
useEffect(() => {  
  // Lógica del efecto  
  
  return () => {  
    // Fase de limpieza (opcional)  
  };  
}, [dependencias]); // El arreglo de dependencias controla cuándo se ejecuta
```

Sección 5: Estado Global: `useContext`

Ideal para evitar el *Prop Drilling* (pasar props a través de muchos niveles de componentes que no las necesitan). Permite que un componente acceda a datos de un "ancestro" lejano.

1. **Crear el contexto:** `const UserContext = createContext();`
2. **Proveer el valor:** `<UserContext.Provider value={data}>`
3. **Consumir el valor:** `const user = useContext(UserContext);`

Sección 6: Referencias y Persistencia: `useRef`

`useRef` devuelve un objeto con una propiedad `.current` que persiste durante todo el ciclo de vida del componente.

- Uso 1: Acceso al DOM. Para hacer foco en un input o medir un elemento.
- Uso 2: Almacenar valores. Guardar información que no debe disparar un nuevo renderizado al cambiar (como un ID de un timer).

Sección 7: Optimización: useMemo y useCallback

Estos Hooks se utilizan para mejorar el rendimiento evitando cálculos o renderizados innecesarios.

- **useMemo:** Memoriza el **resultado** de un cálculo costoso. Solo se recalcula si las dependencias cambian.
- **useCallback:** Memoriza una **instancia de función**. Evita que funciones pasadas como props a componentes hijos optimizados (React.memo) causen re-renders porque su referencia cambia en cada render.

Hook	Qué memoriza
useMemo	El valor devuelto por la función
useCallback	La función misma

Sección 8: Estados Complejos: useReducer

Cuando el estado tiene múltiples subvalores o cuando el próximo estado depende del anterior de forma compleja, useReducer es preferible a useState. Utiliza el patrón *Redux*: una función reductora que recibe el estado actual y una "acción".

Sección 9: Custom Hooks

Son funciones que empiezan con la palabra use y pueden llamar a otros Hooks. Su objetivo es la reutilización de lógica. *Ejemplo:* Un Hook useFetch(url) para manejar la carga de datos en cualquier componente.

Buenas Prácticas y Errores Comunes

1. **No omitas dependencias:** Si usas una variable dentro de `useEffect`, debe estar en el arreglo de dependencias. Usa el linter de React.
2. **No abuses de `useMemo`:** La memorización tiene un costo de memoria. Úsala solo para operaciones realmente pesadas.
3. **Limpieza de efectos:** Siempre cancela suscripciones o timers en la función de *cleanup* para evitar fugas de memoria (*memory leaks*).

Bibliografía y Fuentes Oficiales

1. Documentación Oficial de React: <https://react.dev>
 - o *Referencia de Hooks:* [Built-in React Hooks](#)
 - o *Ciclo de Vida (Preservar y Resetear):* [Preserving and Resetting State](#)
2. MDN Web Docs (JavaScript): [Manipulación del DOM y Closures](#)
3. React Docs Beta (Interactivo): [Describiendo la UI](#)
4. Diagrama Visual del Ciclo de Vida (Legacy vs Modern): [React Lifecycle Methods Diagram](#) (Referencia técnica para entender la transición).